

Guide complet de soutenance – LO07 BlaBlaCar 2026

Partie de Pierre : la couche **Vue** et le visuel (Bootstrap)

Pierre Bheidi (binôme : Moss'Ab Mirande-Ney)

Table des matières

1	Contexte du projet (à connaître absolument)	1
2	Les concepts que je dois savoir expliquer	1
3	Ma partie en détail : les fichiers de la Vue	1
3.1	Le gabarit commun (layout/)	2
3.2	La barre de menu dynamique (le point fort)	2
3.3	Le style Bootstrap utilisé (identique aux TD du prof)	2
3.4	Quelques vues concrètes	2
4	Déroulé : que se passe-t-il quand une page s'affiche ?	3
5	Script de démo (ce que je dis à l'écran)	3
6	Questions potentielles du jury (et réponses)	3

Ce document est un guide complet : même sans avoir codé cette partie, tu pourras tout présenter et répondre aux questions du jury. Lis-le une fois en entier, puis relis le « script de démo » et les « questions/réponses ».

1 Contexte du projet (à connaître absolument)

Le sujet : BlaBlaCar est une application web de **covoiturage**. Les utilisateurs se connectent et, selon leur **rôle**, accèdent à des fonctionnalités différentes :

- **Administrateur** : gère les utilisateurs, les véhicules, les villes.
- **Conducteur** : gère ses véhicules et ses trajets (création, clôture).
- **Passager** : réserve des trajets et consulte ses réservations.

La base de données : 5 tables : **utilisateur** (id, nom, prenom, role, login, password, solde), **vehicule**, **ville**, **trajet** (départ, arrivée, conducteur, véhicule, prix, date, statut actif/passif), **reservation** (lie un trajet et un passager).

L'architecture : Le projet suit le patron **MVC** (Modèle-Vue-Contrôleur) : on sépare les données (Modèle), l'affichage (Vue) et la logique (Contrôleur). **Ma partie, c'est la Vue.**

2 Les concepts que je dois savoir expliquer

MVC : Trois couches. Le **Modèle** parle à la base. Le **Contrôleur** reçoit la requête, appelle le modèle, prépare les données. La **Vue** (ma partie) affiche ces données en HTML. Avantage : on peut changer l'affichage sans toucher à la logique.

Une « vue » : Un fichier PHP qui contient surtout du HTML, avec quelques petites boucles pour afficher une liste. Une vue **ne fait jamais de requête SQL** ni de calcul métier : elle reçoit des données déjà prêtes.

Bootstrap : Une bibliothèque CSS/JS gratuite qui fournit des composants déjà stylés (barre de navigation, cartes, tableaux, boutons). On l'inclut par un lien CDN ; on applique ensuite des **classes** sur nos balises HTML (ex. `class="btn btn-primary"`). On a pris la version **5.3**, la même que dans les TD.

Session : Une mémoire côté serveur qui retient qui est connecté entre les pages (variable `$_SESSION['login_id']`). La Vue la lit pour savoir quel menu afficher.

Échappement (`htmlspecialchars`) : Transformer les caractères spéciaux HTML (<, >) en texte inoffensif avant l'affichage, pour empêcher l'injection de code (faille **XSS**).

3 Ma partie en détail : les fichiers de la Vue

Tous mes fichiers sont dans `projet/app/views/`. Ils sont rangés par rôle : `admin/`, `conducteur/`, `passager/`, `examineur/`, `innovations/`, `auth/`, `home/`, et un dossier `layout/` commun.

3.1 Le gabarit commun (`layout/`)

Trois fragments composent l'ossature de **chaque** page :

- `header.php` : le début du HTML (<head>), le chargement de Bootstrap (CSS + JS par CDN) et un petit bloc de style (le conteneur a une marge haute pour passer sous la barre fixe).
- `fragmentMenu.php` : la **barre de navigation** et le bandeau bleu « Projet BlaBlaCar 2026 » (le *jumbotron*).
- `footer.php` : le pied de page (noms, UTT, date) et la fermeture du HTML.

C'est le contrôleur qui les assemble via une méthode `render()` : on n'écrit donc jamais deux fois l'en-tête.

3.2 La barre de menu dynamique (le point fort)

C'est la pièce centrale de ma partie. Elle :

1. lit l'utilisateur connecté en session ;
2. affiche en haut à gauche les **noms du binôme**, et à droite le **nom de l'utilisateur connecté** avec son **solde** ;
3. affiche **un seul** menu métier selon le rôle (Administrateur, Conducteur *ou* Passager) ;
4. affiche toujours les menus Innovations, Examineur et Se connecter.

```
if ($courant !== null && $courant['role'] === 'administrateur'): ...
elseif ($courant['role'] === 'conducteur'): ...
elseif ($courant['role'] === 'passager'): ...
endif;
```

« Si personne n'est connecté, aucun menu métier n'apparaît » – exactement ce que demande le sujet.

3.3 Le style Bootstrap utilisé (identique aux TD du prof)

- **Barre** : `navbar navbar-expand-lg bg-dark navbar-dark fixed-top` avec des menus déroulants (`dropdown`).
- **Bandeau** : `<div class="mt-4 p-5 bg-primary text-white rounded">` (le *jumbotron* moderne de Bootstrap 5).
- **Contenu** : chaque page est une **carte** (`card / card-body bg-info`) avec un titre, et le contenu dans un bloc intérieur `bg-light p-3 rounded`.
- **Tableaux** : `table table-striped table-bordered table-hover` avec un en-tête rouge `thead table-danger`.

- **Formulaires** : `form-control`, `form-select`, boutons `btn btn-primary`.
- **Messages** : `alert alert-success` (succès) ou `alert-danger` (erreur) ; cartes `text-bg-success` / `text-bg-danger` pour les résultats d'opération.

Tout est en Bootstrap, **sans fichier CSS personnel** (juste un petit bloc de style inline pour le conteneur, comme dans les TD).

3.4 Quelques vues concrètes

- `home/accueil.php` : page d'accueil + comptes de test.
- `auth/login.php` : formulaire de connexion.
- `admin/utilisateurs.php` : tableau de tous les utilisateurs.
- `admin/vehicules.php` : tableau des véhicules. **Attention** : on n'affiche *pas* les clés primaires, et le propriétaire est reconstruit « prénom + nom » – une exigence du sujet.
- `conducteur/trajets.php` : trajets avec un **badge** vert (actif) ou gris (passif).
- `innovations/data.php` : le tableau de bord (statistiques).

4 Déroulé : que se passe-t-il quand une page s'affiche ?

1. L'utilisateur clique sur un lien, ex. `index.php?action=admin_utilisateurs`.
2. Le contrôleur récupère les données (la liste des utilisateurs) auprès du modèle.
3. Il appelle `render('admin/utilisateurs', $data)`.
4. `render()` assemble header + fragmentMenu + ma vue + footer, et `extract($data)` rend la variable `$utilisateurs` disponible dans la vue.
5. Ma vue parcourt `$utilisateurs` avec un `foreach` et génère les lignes du tableau, chaque valeur passée par `htmlspecialchars`.

5 Script de démo (ce que je dis à l'écran)

« Voici la page d'accueil. En haut, la barre de navigation affiche nos deux noms. Tant qu'on n'est pas connecté, seuls trois menus apparaissent : Innovations, Examineur et Se connecter. *[Je me connecte.]* Maintenant qu'on est administrateur, un nouveau menu Administrateur est apparu, et à droite s'affichent le nom et le solde. *[Liste des utilisateurs.]* C'est une carte Bootstrap avec un tableau à en-tête rouge. *[Liste des véhicules.]* Remarquez qu'on n'affiche pas les identifiants techniques : le propriétaire est reconstruit à partir du prénom et du nom. *[Je réduis la fenêtre.]* La barre se replie en menu hamburger : c'est le responsive de Bootstrap. »

6 Questions potentielles du jury (et réponses)

Q : C'est quoi la couche Vue ?

R : La partie qui s'occupe uniquement de l'affichage. Elle reçoit des données déjà préparées par le contrôleur et les met en forme en HTML, sans logique métier ni requête à la base.

Q : Comment la barre de menu sait quel menu afficher ?

R : Elle lit `$_SESSION['login_id']`, récupère l'utilisateur via le modèle, puis teste son champ `role`. Un seul des trois menus métier est affiché selon ce rôle.

Q : Et si personne n'est connecté ?

R : `login_id` vaut -1, donc aucun menu métier n'est affiché : seuls Innovations, Examineur et Se connecter restent visibles.

Q : Pourquoi Bootstrap ?

R : Pour un rendu propre et responsive immédiat, avec des composants testés. C'est aussi la bibliothèque utilisée dans les TD. On garde la maîtrise du HTML et des classes choisies.

Q : Avez-vous écrit du CSS ?

R : Quasiment pas : on utilise les classes Bootstrap. Il y a seulement un petit bloc de style inline dans l'en-tête pour la marge du conteneur sous la barre fixe.

Q : À quoi sert htmlspecialchars ?

R : À échapper les caractères HTML en sortie : c'est une protection contre les failles XSS. Du code injecté dans un champ s'affiche comme du texte au lieu d'être exécuté.

Q : Comment une vue reçoit-elle ses données ?

R : Le contrôleur passe un tableau associatif à `render()`, qui fait un `extract()` : chaque clé devient une variable utilisable dans la vue.

Q : Différence entre `<?=` et `<?php echo` ?

R : `<?= $x ?>` est simplement un raccourci d'écriture pour `echo $x`. On s'en sert pour afficher une valeur dans le HTML.

Q : C'est quoi un fragment / le fragmentMenu ?

R : Un morceau de vue réutilisé sur toutes les pages. Le `fragmentMenu` est la barre de navigation ; on l'écrit une seule fois et il est inclus partout par `render()`.

Q : Comment gérez-vous l'affichage mobile ?

R : Avec les classes responsive de Bootstrap : `navbar-toggler` replie le menu en hamburger, et les tableaux peuvent défiler horizontalement.

Q : Qui gère l'ouverture des menus déroulants ?

R : Le JavaScript de Bootstrap (le bundle chargé dans l'en-tête). On ajoute juste les attributs `data-bs-toggle="dropdown"`.

Q : Pourquoi reconstruire le propriétaire « prénom + nom » dans la liste des véhicules ?

R : Parce que le sujet interdit d'afficher les clés primaires : on ne montre pas l'identifiant du propriétaire mais son nom lisible, obtenu par une jointure faite dans le modèle.

Q : Y a-t-il de la logique métier dans vos vues ?

R : Non. Les vues ne contiennent que des boucles et conditions d'affichage. Tout le métier est dans les contrôleurs et les modèles.